

性能测试工程师

项目全生命周期工作职责

项目背景：购物网站

文档类型	技术评审材料
适用角色	性能测试工程师
项目类型	电商购物网站
文档版本	V1.0

目录

- 一、需求阶段 — 梳理场景、定义指标、了解用户模型
- 二、设计阶段 — 架构评审、测试方案、工具选型
- 三、开发阶段 — 环境搭建、脚本编写、基准测试、数据准备
- 四、测试阶段 — 分层执行、监控定位、分析调优、报告输出
- 五、总结 — 核心理念与价值

一、需求阶段

核心任务：把模糊的业务期望转化为可量化的性能指标。

1.1 梳理关键业务场景并确定性能目标

与产品、业务方共同识别出对性能最敏感的核心场景。对于购物网站，通常包括：

- 首页加载 — 用户的第一印象，直接影响跳出率
- 商品搜索/列表页 — 高频操作，涉及复杂数据库查询与排序
- 商品详情页 — 承载大量图片和动态数据（库存、价格、评论）
- 加入购物车 — 写操作，涉及并发冲突控制
- 下单支付 — 最核心的业务链路，涉及库存扣减、支付网关回调
- 促销/秒杀场景 — 瞬时流量洪峰，对系统韧性的极限考验

1.2 定义可量化的性能指标（SLA/SLO）

推动团队明确以下量化指标，杜绝"要快""不能卡"等模糊描述：

性能指标	定义	示例目标
平均响应时间（Avg RT）	请求从发出到 收到响应的平均耗时	首页 < 1.5s 搜索 < 2s 下单 < 3s
TP99 响应时间	99% 的请求 响应时间上限	各接口 TP99 不超过 平均值的 3 倍
吞吐量（TPS）	系统每秒处理的 事务数	支持 500 并发 用户同时在线
错误率	在目标并发下的 请求失败比率	< 0.1%
秒杀峰值	瞬时最大请求 处理能力	瞬时 5000 QPS 系统可降级不崩溃

1.3 了解用户模型

调研或与业务方确认：日活用户量、高峰时段分布、用户行为比例。这些是后续设计压测模型的基础数据。

行为类型	占比	说明
浏览	70%	首页、列表、详情等页面浏览
搜索	20%	关键词搜索、筛选、排序

行为类型	占比	说明
加购	8%	将商品加入购物车
下单支付	2%	提交订单并完成支付

二、设计阶段

核心任务：提前识别架构中的性能风险，并制定完整的测试策略。

2.1 参与架构评审，提出性能视角的建议

数据库设计

- 商品表、订单表是否有合理索引？
- 库存扣减采用什么方案（乐观锁 / Redis 预扣减）？
- 大表是否考虑分库分表？

缓存策略

- 热门商品是否走 Redis 缓存？
- 缓存击穿、雪崩有无防护机制（互斥锁、永不过期策略）？

架构瓶颈

- 单体还是微服务？服务间调用是同步还是异步？
- 下单链路是否引入消息队列进行削峰填谷？

第三方依赖

- 支付网关的超时和熔断设置是否合理？
- 外部 API 的 SLA 是否满足系统整体性能目标？

2.2 编写性能测试方案

性能测试方案是后续执行的蓝图，核心内容包括：

方案内容	详细说明
测试范围	明确哪些接口要测、哪些不测及原因
测试类型规划	基准测试 → 负载测试 → 压力测试 → 稳定性测试 → 秒杀专项测试
测试环境要求	环境规格、与生产的差异比（如 1:4）
数据准备策略	所需数据量级（商品、用户、历史订单）
压测模型设计	各场景用户比例、思考时间、递增策略

2.3 评估测试工具选型

工具	语言	适用场景	特点
JMeter	Java	通用 HTTP/RPC	GUI 操作，插件丰富
Gatling	Scala	高并发 HTTP	DSL 脚本，报告美观
Locust	Python	灵活场景编排	Python 脚本，易扩展
k6	JavaScript	轻量级 CI 集成	CLI 工具，云原生友好

三、开发阶段

核心任务：做好一切准备工作，确保开发完成后能立即开测。

3.1 搭建性能测试环境

- 协调运维搭建独立的性能测试环境（绝不在生产环境直接压测）
- 部署监控体系：Prometheus + Grafana 监控服务器资源
- 部署链路追踪系统（SkyWalking / Jaeger）用于定位慢调用

3.2 编写性能测试脚本

开发阶段接口文档逐步输出，同步编写测试脚本，重点关注：

- 参数化：用户账号池、商品 ID 池、收货地址等全部做参数化处理
- 关联处理：登录获取 Token → 搜索 → 取商品 ID → 加购 → 下单，整条链路的上下文串联
- 数据隔离：压测数据需有特殊标识，方便事后清理，避免污染真实业务数据

3.3 配合开发做单接口基准测试

当核心接口（如下单）开发完成后，先做单接口的基准测试，尽早暴露明显的性能问题（如 SQL 未走索引、N+1 查询等）。此阶段修复成本最低。

3.4 准备压测数据

数据准备是容易被低估但极其重要的工作，需提前规划：

数据类型	目标量级	用途
商品数据	10 万+	模拟真实商品库规模
用户数据	5 万+	提供足够的并发账号池
历史订单	100 万+	模拟生产环境表体量
地址数据	5 万+	下单流程参数化

四、测试阶段

核心任务：执行、分析、调优、验证的循环迭代过程。

4.1 按计划分层执行测试

测试类型	目标	方法
基准测试	确认功能正确 拿到性能基线	单接口、单用户执行
单场景负载测试	找到每个接口的 性能拐点	逐步加压 50→100→200→500 并发
混合场景负载测试	验证系统 整体承载能力	按用户模型混合多场景 浏览70%+搜索20%+下单10%
压力测试	观察极限下的 表现和恢复能力	超过目标并发继续加压 直到系统出现拐点
稳定性测试	发现内存泄漏 连接池耗尽等	目标负载 70%-80% 运行 8-24 小时
秒杀专项测试	验证限流/降级 /队列机制	模拟瞬间大量请求涌入

4.2 监控和定位瓶颈

压测过程中需同时关注多个层面：

监控层面	关注指标
应用层	响应时间、TPS、错误率、线程池使用情况
中间件层	Redis 命中率、MQ 堆积量、连接池使用率
数据库层	慢 SQL、锁等待、连接数、主从延迟
系统层	CPU、内存、磁盘 I/O、网络带宽
链路层	通过分布式追踪找到调用链中最慢环节

4.3 分析瓶颈并推动优化

以下是一个购物网站中常见的实际案例：

压测发现下单接口在 200 并发时 TP99 飙升到 8 秒。通过链路追踪定位到库存扣减环节的行锁争用严重。建议开发将库存预扣减迁移到 Redis，数据库做异步扣减，并提供压测数据作为决策依据。

4.4 回归验证

优化后重新执行相同的测试场景，对比优化前后的数据，确认问题已解决且没有引入新的性能回退。

4.5 输出性能测试报告

最终报告应包含以下核心内容：

测试结论：系统是否满足既定性能指标

各场景详细数据：响应时间、TPS、错误率的完整记录

瓶颈分析：发现的性能问题及处理结果

系统容量评估：当前架构可支撑的最大业务量

扩容建议：未来业务增长的资源规划建议

上线监控关注点：需要在生产环境持续监控的指标

五、总结

性能测试工程师的价值不在于会用什么工具，而在于全程参与、提前介入、用数据说话、推动问题闭环。

阶段	核心价值	关键产出
需求阶段	将模糊期望量化为指标	性能指标定义、用户模型
设计阶段	提前识别架构性能风险	性能测试方案、风险清单
开发阶段	并行准备确保无缝衔接	测试环境、脚本、压测数据
测试阶段	执行验证推动调优闭环	性能测试报告、优化建议

越早介入，发现问题的修复成本越低。在购物网站这类直接影响用户体验和交易转化率的项目中，性能测试做得好不好，直接关系到上线后是否会出现"一搞活动就崩"的窘境。